

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА**

**ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №1
«Організація професійного Python-проєкту та Git»
з курсу «Професійний Python»
Варіант №4 — «Система банківських рахунків»**

Виконав(ла): Фрадкова А.С.

Група: ФЕП-32

Перевірів(ла): доц. Куньо І.М.

Дата: 15.09.2026

1. Мета роботи

Ознайомитися з основними принципами організації професійного Python-проєкту, навчитися створювати ізольоване середовище розробки, формувати структуру проєкту за принципом src-layout, працювати з модулями та пакетами Python, створювати файл pyproject.toml, використовувати Git для контролю версій та оформлювати базову документацію проєкту.

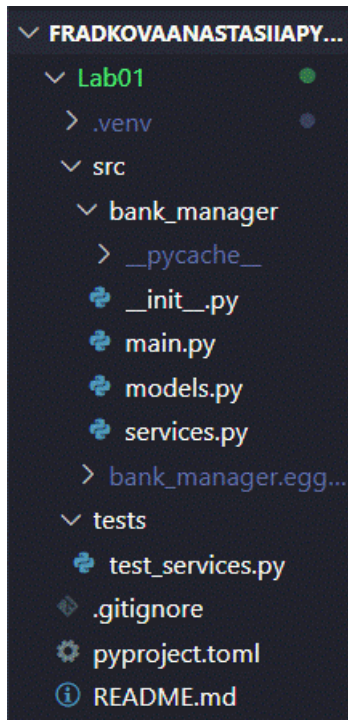
2. Індивідуальний варіант і постановка задачі

Варіант №4 — «Система банківських рахунків». Для кожного рахунку зберігати ім'я клієнта, номер рахунку, баланс. Реалізувати поповнення рахунку, списання коштів, перевірку балансу, пошук рахунку, обчислення загальної суми коштів. Не дозволяється отримувати від'ємний баланс.

Проєкт повинен мати src-layout, щонайменше три Python modules, data model, окремий module з business logic, main.py, type hints, README.md, .gitignore, pyproject.toml і Git repository.

3. Структура проєкту

```
Lab01/
├── pyproject.toml
├── README.md
├── .gitignore
├── src/
│   ├── bank_manager/
│   │   ├── __init__.py
│   │   ├── main.py
│   │   ├── models.py
│   │   └── services.py
├── tests/
│   └── test_services.py
```



Каталог `src/bank_manager` містить пакет застосунку, `models.py` описує модель даних `Student`, `services.py` містить business logic, `main.py` є точкою входу, а `tests/test_services.py` містить автоматизовані тести.

4. Опис створених модулів

4.1. models.py

Для представлення банківського рахунку створено датаклас `Account` з полями `number`, `client_name` і `balance`.

```
from dataclasses import dataclass

@dataclass(frozen=True, slots=True)
class Account:
    number: int
    client_name: str
    balance: float
```

4.2. services.py

У модулі реалізовано функції `top_up()`, `withdraw()`, `check_balance()`, `find_account()`, `calculate_total_sum()`. Бізнес-логіка не взаємодіє з консоллю.

```
from bank_manager.models import Account
from dataclasses import replace

def top_up(account: Account, sum: float) -> Account:
    return replace(account, balance=account.balance + sum)

def withdraw(account: Account, sum: float) -> Account | None:
    return None if account.balance < sum else replace(account,
balance=account.balance - sum)

def check_balance(account: Account) -> float:
    return account.balance

def find_account(accounts: list[Account], number: int) -> Account |
None:
    for account in accounts:
        if account.number == number:
            return account
    return None

def calculate_total_sum(accounts: list[Account]) -> float:
    return sum(account.balance for account in accounts)
```

4.3. main.py

Модуль `main.py` створює демонстраційні дані, викликає функції з `services.py` та виводить результати. Конструкція `if __name__ == "__main__":` відокремлює точку входу від функцій програми.

```
from bank_manager.models import Account
from bank_manager.services import top_up, withdraw, check_balance,
find_account, calculate_total_sum
```

```
def create_accounts() -> list[Account]:
    return [
        Account(number=1, client_name='Alice', balance=25.5),
        Account(number=2, client_name='Bob', balance=10.0)
    ]

def print_account_info(account: Account) -> None:
    print(f"Name: {account.client_name}, number: {account.number}, balance: {account.balance}")

def main() -> None:
    accounts = create_accounts()

    print("INITIAL BALANCES")
    for account in accounts:
        print_account_info(account)
    print()

    print("TOP UP")
    accounts[0] = top_up(accounts[0], 3.0)
    print_account_info(accounts[0])
    print()

    print("FAILED WITHDRAWAL")
    withdrawal = withdraw(accounts[0], 100.0)
    if not withdrawal:
        print("Error! Balance cannot be negative.")
        print_account_info(accounts[0])
    print()

    print("SUCCESSFUL WITHDRAWAL")
```

```
withdrawal = withdraw(accounts[0], 3.0)

if withdrawal:
    accounts[0] = withdrawal
    print_account_info(accounts[0])
print()

print("ACCOUNT SEARCH")
target_number = 2
account = find_account(accounts, target_number)
if account:
    print(f"Found account {target_number}.")
    print_account_info(account)
else:
    print(f"Account {target_number} not found.")
print()

print("TOTAL SUM CALCULATION")
print(f"Total sum: {calculate_total_sum(accounts):.2f}")

if __name__ == "__main__":
    main()
```

5. Файл pyproject.toml

Файл задає build system setuptools, основні метадані проєкту (назву, версію та опис), вимогу до версії Python 3.11+ та console script bank-manager для запуску програми.

```
Lab01 > ⚙ pyproject.toml
1  [build-system]
2  requires = ["setuptools>=70"]
3  build-backend = "setuptools.build_meta"
4
5  [project]
6  name = "bank-manager"
7  version = "0.1.0"
8  description = "Bank accounts management laboratory project"
9  requires-python = ">=3.11"
10
11 [project.scripts]
12 bank-manager = "bank_manager.main:main"
```

6. Файл .gitignore

3 Git виключено virtual environment, кеші, IDE configuration, build artifacts і .env.

```
Lab01 > 💎 .gitignore
1  .venv/
2  __pycache__/
3  *.pyc
4  .pytest_cache/
5  .idea/
6  .vscode/
7  dist/
8  build/
9  *.egg-info/
```

7. README.md

```
# Bank Manager (Lab 01)

## Description

Console application for managing bank accounts and financial operations.
```

```

## Requirements

Python 3.11+

## Installation

python -m venv .venv

.venv\Scripts\Activate.ps1

python -m pip install -e .

## Run

python -m bank_manager.main

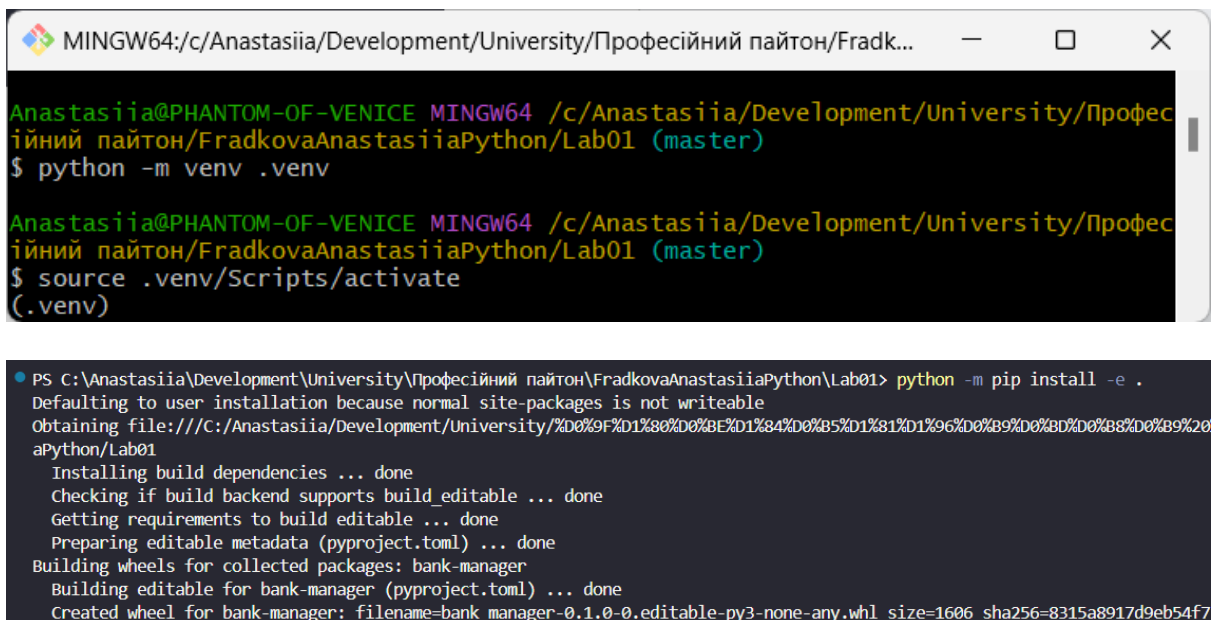
## Author

Full name: Anastasiia Fradkova

Group: FEP-32

```

8. Створення virtual environment та запуск



```

MINGW64:/c:/Anastasiia/Development/University/Професійний пайтон/FradkovaAnastasiiaPython/Lab01
Anastasiia@PHANTOM-OF-VENICE MINGW64 /c:/Anastasiia/Development/University/Професійний пайтон/FradkovaAnastasiiaPython/Lab01 (master)
$ python -m venv .venv

Anastasiia@PHANTOM-OF-VENICE MINGW64 /c:/Anastasiia/Development/University/Професійний пайтон/FradkovaAnastasiiaPython/Lab01 (master)
$ source .venv/Scripts/activate
(.venv)

PS C:\Anastasiia\Development\University\Професійний пайтон\FradkovaAnastasiiaPython\Lab01> python -m pip install -e .
Defaulting to user installation because normal site-packages is not writeable
Obtaining file:///C:/Anastasiia/Development/University/Професійний пайтон/FradkovaAnastasiiaPython/Lab01
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Building wheels for collected packages: bank-manager
Building editable for bank-manager (pyproject.toml) ... done
Created wheel for bank-manager: filename=bank_manager-0.1.0-0.editable-py3-none-any.whl size=1606 sha256=8315a8917d9eb54f7

```

9. Результат роботи програми

```
PS C:\Anastasiia\Development\University\Професійний пайтон\FradkovaAnastasiiaProfPython> python -m bank_manager.main
INITIAL BALANCES
Name: Alice, number: 1, balance: 25.5
Name: Bob, number: 2, balance: 10.0

TOP UP
Name: Alice, number: 1, balance: 28.5

FAILED WITHDRAWAL
Error! Balance cannot be negative.
Name: Alice, number: 1, balance: 28.5

SUCCESSFUL WITHDRAWAL
Name: Alice, number: 1, balance: 25.5

ACCOUNT SEARCH
Found account 2.
Name: Bob, number: 2, balance: 10.0

TOTAL SUM CALCULATION
Total sum: 35.50
PS C:\Anastasiia\Development\University\Професійний пайтон\FradkovaAnastasiiaProfPython> █
```

10. Робота з Git

У проєкті було створено 4 коміти:

```
git init
```

```
git add .
```

```
git commit -m "Create Lab01 project structure"
```

```
git add .
```

```
git commit -m "Add account model and business logic services"
```

```
git add .
```

```
git commit -m "Add application entry point"
```

```
git add .
```

```
git commit -m "Add documentation"
```

10.1. Результат git status

```
MINGW64:/c/Anastasiia/Development/University/Професійний пайтон/Fradk...
(.venv)
Anastasiia@PHANTOM-OF-VENICE MINGW64 /c/Anastasiia/Development/University/Професі
ійний пайтон/FradkovaAnastasiiaPython (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
(.venv)
```

10.2. Результат git log --oneline

```
MINGW64:/c/Anastasiia/Development/University/Професійний пайтон/Fradk...
Anastasiia@PHANTOM-OF-VENICE MINGW64 /c/Anastasiia/Development/University/Професі
ійний пайтон/FradkovaAnastasiiaPython (main)
$ git log --oneline
a660080 (HEAD -> main, origin/main) Add documentation
7d5d029 Add application entry point
c0ed119 Add Account model and business logic services
88b47af Create Lab01 project structure
(.venv)
Anastasiia@PHANTOM-OF-VENICE MINGW64 /c/Anastasiia/Development/University/Професі
```

12. Висновок

Ознайомилася з основними принципами організації професійного Python-проєкту, навчилася створювати ізольоване середовище розробки, формувати структуру проєкту за принципом src-layout, працювати з модулями та пакетами Python, створювати файл pyproject.toml, використовувати Git для контролю версій та оформлювати базову документацію проєкту.